

Authorization / RBAC / Super-Admin / Token-Scoping Model — To-Be Design

Revision: 1 Last modified: 2026-07-10T11:18:54Z

Scope. This is the AUTHORIZATION design for the multi-account Helix OTA control plane (server/, Go/Gin modular monolith). It owns **OQ-2** (the permission-model shape — 00_INDEX §5) that 20_target_multitenancy_data_model.md deliberately deferred here. It designs: the permission model (RBAC vs ABAC vs role+scope hybrid), the sign-in + account-selection authZ flow, token scoping (the account_id / project claim), the app-layer-middleware + DB-layer-RLS enforcement stack, the single canonical role vocabulary (§11.4.186 divergence reconciliation), and least-privilege + revocation.

SSOT deference. 20_target_multitenancy_data_model.md is the **single source of truth** for the Account / User / Membership / Role/Permission entity shapes. This doc **defers to and cites it** — **it never invents a conflicting entity shape** (§11.4.186). Where a field is named here it is quoted from 20_* (or from the shipped code) and marked as such; the authorization *behaviour* over those entities is this doc's to design.

Reading order. 00_INDEX.md (mandate + OQ-2) →
10_existing_auth_and_project_model.md (as-is auth facts, every file:line grounded there) →
20_target_multitenancy_data_model.md (entity SSOT) →
12_existing_ui_surfaces_and_design_system.md (the role divergences §5 reconciles). Sibling wiring:
23_ui_ux_all_surfaces.md (sign-in/switcher UI),
24_project_side_cli.md (CLI API-key scope),
25_device_side_update_client.md (device token scope),
26_security_threat_model.md (owns the cross-account-leakage paired-mutation proof), 22_api_surface.md (owns the endpoint shapes).

Every open choice is a **recommendation-with-tradeoffs for operator decision**, never a silent pick (§11.4.6 / §11.4.66). This is a **design proposal, not implemented** (§11.4.6 honest boundary at the foot).

1. Permission model (OQ-2 — “ultimate flexibility to the maximum”)

The mandate (00_INDEX §1 item 6) wants “ultimate flexibility to the maximum”: the super-admin assigns “who can access / do what” per account. OQ-2 is *which* model delivers that. Three canonical models, then a recommendation.

1.1 The three candidate models

Model	What it is	Fit for Helix OTA	Cost
RBAC (tenant-scoped)	fixed roles, each membership carries a role <i>within one account</i> ; a resource×action → min-role matrix decides access	maps 1:1 onto the shipped ROLE_HIERARCHY (clients/ota-manager/src/lib/permissions.ts:12-18) + the RESOURCE_PERMISSIONS grid (permissions.ts:30-42) AND onto 20_* Membership.role CHECK IN ('viewer', 'operator', 'admin')	lowest; caps flexibility at a fixed role set
ABAC (policy engine)	access is a policy over <i>attributes</i> of (user, resource, environment) evaluated by an engine (Cedar / OPA / Cerbos)	maximum expressiveness; but introduces a policy-engine dependency + a policy language the whole team must own	highest; heaviest to build/test/operate
Role+scope hybrid	RBAC base + a mandatory tenant-isolation scope predicate at the top of every decision + a thin ABAC attribute-override layer (e.g. suspended-	the industry default for multi-tenant B2B (AWS, Auth0, Cerbos — \$Sources) — “roles are coarse; attributes cover the exceptions”	moderate; grows into ABAC only where needed

Model	What it is	Fit for Helix OTA	Cost
	account / locked-user deny)		

Why not pure RBAC alone: it forces every exception into a new role, and roles “explode ... you end up with hundreds of roles per tenant” (Cerbos — §Sources). **Why not pure ABAC now:** a full policy engine is over-scoped for a control plane whose entire permission surface today is one coarse `requireRole` check per route (10_* §2) — it would be a large new dependency before a single tenant exists.

1.2 Recommendation — RBAC-first role+scope hybrid, ABAC-capable by growth (not day-one)

Recommend the role+scope hybrid, delivered RBAC-first, composed as three stacked decisions evaluated in this fixed order (AWS’s “keep tenant isolation at the top” + Auth0’s “roles checked within the tenant context” — §Sources):

- 1. Tenant-isolation predicate (always first).** The principal’s active `account_id` (§3 claim) **MUST** equal the resource’s `account_id`; a super-admin (§1.4) is the only principal that bypasses this. This is the non-negotiable top gate — AWS Verified Permissions expresses exactly this as `resource in principal.Tenant` on *every* policy (§Sources). It is the authZ-layer twin of 20_*’s data-layer RLS (§4).
- 2. RBAC role check.** Within the account, the membership role (`viewer < operator < admin`, 20_* §1.4, mirroring the shipped `ProjectRole` rank-compare `handlers_project.go:78-92`) must meet the `resource×action` minimum from the permission matrix (§1.3).
- 3. ABAC attribute-override (deny-only, thin).** A small closed set of attribute conditions can only ever *subtract* access: `account status='suspended'` (20_* §1.1), `user is_active=false` (20_* §1.2), or an API-key `revoked_at IS NOT NULL` (§6). This is the AWS `account_lockout_flag` pattern (§Sources) — attributes that DENY, never a parallel grant path.

The “ultimate flexibility” growth seam (no second data-model change). 20_* §1.4 already emits the RBAC skeleton now (`Membership.role` enum) **and** ships the reserved roles / permissions / `role_permissions` tables as commented DDL. When an account genuinely needs custom roles beyond the three, this doc’s model escalates by turning on those tables — `Membership.role` becomes a `role_id` FK, and the `resource×action` matrix (§1.3) moves from a fixed map into `role_permissions` rows — WITHOUT touching the

enforcement stack (§4) or re-migrating. That is the “maximum flexibility” the mandate asks for, reachable additively (20_* §1.4 tradeoff). **Full ABAC (arbitrary per-user resource:action grants via an engine) stays a documented future escalation**, not built now.

Tradeoffs of the recommendation. RBAC-first reuses the entire shipped role-hierarchy + matrix and the existing rank-compare tooling verbatim (smallest build, least risk), and the tenant-isolation-first ordering closes the cross-account hole by construction. The cost: custom-per-account roles and true attribute policies are a later escalation, not available at first ship. If the operator needs arbitrary custom roles on day one, escalate straight to the 20_* reserved-tables path (still no engine); if true policy-as-code is a launch requirement, adopt Cedar/OPA — heavier, flagged for operator decision (§11.4.66).

1.3 The per-account roles + the permission set (resource × action)

Per-account roles (RBAC, from 20_* §1.4 — cited, not redefined):
viewer < operator < admin, carried on Membership.role per (user, account). The same user is admin in account A and viewer in account B — the whole point of per-account scoping (20_* §1.3; Auth0 “admin in Tenant A, viewer in Tenant B” — §Sources).

The permission set = the shipped RESOURCE_PERMISSIONS grid, promoted to server-authoritative. The grid already exists as the exact resource×action → min-role matrix (clients/ota-manager/src/lib/permissions.ts:30-42) — but **today it is client-side UX only**; the server enforces solely the coarse per-route requireRole (10_* §2, middleware.go:79-97), so the grid is advisory, not a security boundary. This design **promotes that same matrix to the server as the authoritative permission set** (the RBAC step §1.2(2) evaluates it), keeping the client copy as UX pre-filtering only:

Resource	view	create	update	delete
devices	viewer	operator	operator	admin
artifacts	viewer	operator	operator	admin
deltas	viewer	operator	admin	admin
releases	viewer	operator	operator	admin
deployments	viewer	operator	operator	admin
rollouts	viewer	operator	operator	admin
groups	viewer	operator	operator	admin

Resource	view	create	update	delete
group_members	viewer	operator	operator	operator
audit	admin	admin	admin	admin
telemetry	viewer	device	admin	admin
projects	viewer	admin	admin	super_admin

(Verbatim from `permissions.ts:30-42`; `device` and `super_admin` appear as cell minimums and are handled by §1.4 + §5 — `telemetry.create=device` is the device-token path, `projects.delete=super_admin` is the global-flag path.)

Recommendation: adopt this grid unchanged as the server-side permission set for the RBAC step, so client and server share one matrix (a §11.4.186 single-representation win). **Tradeoff:** the grid is a *minimum-role* model (coarse) — fine for the RBAC-first ship; per-permission granularity is the §1.2 growth seam.

1.4 Super-admin = a GLOBAL identity flag, not a per-account role

Per 20_* §6 (cited, not redefined): the super-admin is the **global** `users.is_super_admin boolean` — a property of the *identity*, NOT a membership role, and NOT a value of `Membership.role`. A super-admin need not be a member of any account and **bypasses the §1.2(1) tenant-isolation predicate** entirely (sees/controls every account — 00_INDEX §1 item 4). This generalizes the *exact precedent already in the code*: a global admin bypasses the per-project ACL and implicitly reaches every project (`handlers_project.go:43-60`; `list-all :162-199`). `is_super_admin` is settable **only via config/env bootstrap, never a request** (20_* §6; the same trust boundary as `resolvePublicKey`, `handlers_artifact.go:274-283`). Its blast radius, auditability, and revocation are §4/§6; the cross-account-leakage proof is 26_*'s.

2. Sign-in + account-selection authorization flow

Both SPAs already ship a working sign-in + session + RBAC (12_* §4). The multi-account delta is **which account a session is acting in**, and how the server proves (`account`, `project`, `role`) on every route — today gated only by the global role (10_* §2).

2.1 The flow (identity token → account-scoped access token exchange)

The shipped flow: POST /api/v1/auth/login → handleLogin authenticates via the injected UserDirectory and calls issueTokenPair (handlers_auth.go:77-99,123-137), returning a short-lived access token + an opaque, single-use-rotating, in-memory refresh token (handlers_auth.go:28-72). The multi-account flow keeps that and inserts an **account-selection exchange** (the WorkOS pattern 23_/24_/25_* already reference):

1. LOGIN (identity) POST /auth/login user+pass → identity access token (roles, NO account_id)

+ identity-scoped

refresh token (unchanged shipped flow)
2. RESOLVE MEMBERSHIPS server: ListUserAccounts(userID) (20_* §3.1) → the user's accounts[]
3. SELECT ACCOUNT client: pick active account (23_* §2 – auto-select if exactly 1)
4. EXCHANGE (account-scope) POST /auth/token-exchange identity token + chosen account_id

→ short-lived access

token WITH account_id claim (§3)
5. ACT every OTA request carries the account-scoped bearer on the existing hot path
6. SWITCH re-exchange for a different account_id (23_* §2.3); refresh token stays identity-scoped

- **Super-admin** skips account selection → the exchange (or login) yields a **System-scope** token (no single account_id, or an explicit super-admin marker) that bypasses §1.2(1) and lands on the global console (23_* §1.1). An explicit “act within account X” affordance re-exchanges for a specific account_id so even super-admin actions are attributable to one tenant when scoped (audit §4).
- **Regular user, 0 memberships** → the exchange returns no scope → fail-closed dead-end (23_* §2.1); never a silent unscoped session.

2.2 Recommendation — on-demand exchange, membership re-verified per request

Recommend the on-demand exchange (step 4): the account claim is **derived at selection time, never embedded in the long-lived credential** (WorkOS — §Sources; 23_* §1.3). This composes with 20_* §2.2 #7's **recommendation** (resolve membership per-request to

support account switching without re-minting) as a deliberate **belt-and-suspenders**: the `account_id` claim is stamped at exchange (the belt — fast, no per-request account lookup on the hot path for the common case), AND `requireAccountAccess` (§4) **re-verifies the `GetAccountMembership(userID, account_id)` row on every request** (the suspenders — a stale/forged claim cannot outlive the membership). This is the reconciliation of the two docs' framings: 20_* says "resolve per-request"; this doc says "stamp the claim AND still verify per-request" — the claim is a routing hint, the membership row is the authority.

Tradeoffs. On-demand exchange costs one extra round-trip at account-selection/switch (24_* Option B con) and requires the `account_id` claim in Claims (§3). The alternative — embed a long-lived multi-account token — is rejected: a compromised or rotated session could then act in the wrong account, and lateral movement across a flat multi-account token is "the pattern to avoid" (multi-tenant-saas.com — §Sources). The per-request membership re-verify adds one indexed lookup per request (`GetAccountMembership`, indexed 20_* §4) — accepted as the price of not trusting a claim snapshot.

2.3 Enforcing (account, project, role) on every OTA route

Today OTA routes (`device/artifact/release/deployment/group/telemetry`) are gated ONLY by the global `requireRole` (`server.go:192-242`) and carry no tenant gate — the load-bearing gap (10_* §2). The design wires a middleware chain onto every scoped route:

`authMiddleware` (verify token, extract `account_id` claim — `middleware.go:58-77`) → **`requireAccountAccess(minRole)`** (NEW — verify `GetAccountMembership`, reject suspended account/inactive user, put `{AccountID, ProjectID}` scope in context) → `requireProjectAccess` (existing pattern `handlers_project.go:37-74`, now wired onto ALL resource routes, keyed on the resource's `project_id` from 20_* §2) → the coarse `requireRole` remains as the outer role gate. `requireAccountAccess` is the new analogue of `requireProjectAccess`, one level up (10_* §8 seam #8).

3. Token scoping (extend the {sub, roles, iat, exp} claim set)

3.1 The new claim

Extend the shipped Claims{sub, roles, iat, exp} (token.go:31-36) with an **account_id** claim. Per multi-tenant-saas.com (§Sources): make it a **custom, top-level, immutable claim** (not a reused standard OIDC field), **validated at the one gateway** (authMiddleware — the only place raw bytes become a trusted account_id) and read by every downstream layer from that verified payload only — never from a spoofable header (the same trust boundary as the device claim, 25_* §1.2, and resolvePublicKey).

Claim	Source	Notes
sub	shipped (token.go:32)	subject/username → resolves to user_id (20_* §1.2)
roles	shipped (token.go:33)	canonical vocabulary §5; carries super_admin when is_super_admin
iat / exp	shipped	short access TTL (15 min default, config.go) bounds staleness (§Sources, §6)
account_id	NEW	the active account; absent = unscoped (§3.3); super-admin System-scope = absent-or-marker (§2.1)
project_id	NEW, device/CLI only	device tokens (25_* §1.2) + optionally CLI keys (24_* §1.1) narrow to one project

Device + CLI tokens additionally carry project_id: the device token stamps (account_id, project_id) at registration (mintDeviceToken, server-minted/server-verified — 25_* §1.2), and the CLI's

account+project-scoped API key resolves to (account_id, project_id, permissions) — either directly per call or via a token-exchange (24_* §1.3, the same exchange as §2.1). Admin/operator/viewer *user* tokens carry account_id and select the project via the request path/context, not a token claim.

3.2 Signing stays config-only (§11.4.10)

Adding claims does NOT touch the trust boundary: the HMAC-SHA256 signing secret continues to come ONLY from config (HELIX_TOKEN_SECRET, config.go:66-70,178-184), never from a request, and the two-part sign/verify (token.go:60-102) is unchanged — a larger payload, same symmetric MAC. The dev-fallback secret remains the known prod risk (10_* §1) — out of scope here, flagged. **No per-tenant signing key** is proposed for MVP (one global secret, simplest); per-tenant signing-key rotation (multi-tenant-saas.com — §Sources) is a documented hardening escalation 26_* may take, not built now.

3.3 Backward-compatibility for legacy tokens (no account_id claim)

A token minted before the cutover (or by the current code path) has no account_id. The account_id claim **MUST** be **optional in the struct** so legacy tokens still *parse and verify* (mirrors 20_* — legacy fixtures WITHOUT new fields must still parse). Three handling options at the enforcement layer:

- **Option A — fail-closed (RECOMMENDED).** A token with no account_id claim is denied on every account-scoped route (requireAccountAccess finds no scope → 403), EXCEPT a super_admin token (which is legitimately account-unscoped, §2.1). The user simply re-authenticates + selects an account to get a scoped token; the 15-min access TTL means legacy tokens age out within one TTL window anyway.
- **Option B — default-account fallback.** Treat a missing account_id as the seeded __default__ account (20_* §5 Option A). *Con:* silently grants the default account's scope to any legacy token — an isolation-softening special case littering every read path (the exact NULL-scoping hole 20_* §5 rejects).
- **Option C — reject outright.** Any token lacking account_id fails verification. *Con:* a hard flag-day; needlessly breaks in-flight refresh during the cutover.

Recommendation: Option A (fail-closed on scoped routes, super-admin exempt). It never widens scope (Option B’s fault), never hard-breaks the cutover (Option C’s fault), and self-heals within one short TTL. It composes with **20_* §5 Option A** (default-account backfill): once every row has a non-NULL `account_id`, a legacy unscoped token has nothing to legitimately act on anyway. **Tradeoff:** users with a live legacy token must re-select an account once — a one-time, self-clearing friction.

4. Enforcement layers — app-layer middleware + DB-layer RLS

Two independent layers that compose as **defense in depth** — a forgotten scope in one is caught by the other. This is the authorization realization of **20_* §0/§6**.

Layer	Mechanism	What it enforces	What it catches
L1 App — middleware	<code>authMiddleware</code> → <code>requireAccountAccess</code> → <code>requireProjectAccess</code> → <code>requireRole</code> (§2.3)	the §1.2 3-step decision: tenant-isolation predicate, RBAC matrix, attribute deny-override	a request whose token account target, or whose role < matrix minimum or whose account is suspended
L2 App — scoped store	explicit <code>accountID</code> param on <code>get/create/update</code> + <code>AccountID/ProjectID</code> on <code>*Filter</code> structs (20_* §3.2 hybrid)	every <code>store.Repository</code> read/write carries scope — a missed scope is a compile error , not a runtime leak	a handler that forgot to pass scope (build fails)
L3 DB — RLS	<code>pgx</code> sets <code>app.current_account</code> GUC per request; <code>CREATE POLICY ... USING (account_id =</code>	database-enforced row scoping —	a bug that slipped L1+L2 —

Layer	Mechanism	What it enforces	What it catches
	<code>current_setting('app.current_account', true))</code> (20_* §4/§6)	“works even when your application code has bugs” (AWS — §Sources)	the DB still refuses cross-account rows

How they compose. L1 decides *may this principal act*; L2 makes *scope non-optional in the code*; L3 makes *scope non-optional in the database*. A super-admin bypasses L1’s tenant predicate (§1.4) and L3 via a **policy-based bypass** — a super-admin GUC the RLS USING clause also honors — NOT a BYPASSRLS role, because AWS is explicit that policy-based admin access “keeps admin access auditable and revocable” whereas a BYPASSRLS role is invisible to the policy (20_* §6 — cited). The MemoryRepository (the default MVP backend, `main.go:80`) has no RLS; there L1+L2 are the whole guarantee, so the explicit-param L2 (compile-time scope) is what makes the memory backend safe — flagged: **RLS is a Postgres-only second layer; the memory backend leans entirely on L1+L2** (26_* must test both backends).

4.1 Audit of super-admin (and all scoped) actions

The shipped audit records only ActorSubject, never `account_id`, and UserID is always empty (`handlers_audit.go:37-46`; `audit_wire.go:41-43`; 10_* §6). This design:

- **Populates `account_id` (the affected tenant) + `project_id` + `UserID`** on every audited mutation (20_* §2.1) — the audit trail becomes tenant-attributable, and `handleListAudit` (admin-only, `server.go:242`) gains an account filter.
- **Tags super-admin actions explicitly.** Because the DB bypass is **policy-based, not a BYPASSRLS role**, every super-admin query still names the account it touched — so a super-admin acting “within account X” (§2.1) writes an audit row naming X + the super-admin `user_id`. A cross-account super-admin action is attributable per query, not an anonymous god-mode (20_* §6; the break-glass “every use logged + alerted” principle — §Sources).

Recommendation: treat super-admin like a **break-glass / emergency-access identity** (§Sources) — emergency-and-administration-only, isolated from normal admin, **every use audited with the affected `account_id`**, credentials config-seeded + rotatable (§6). A super-admin

action with no audit row naming the affected tenant is a §11.4 audit-layer bluff. **Tradeoff:** full break-glass rigor (real-time alerting on every super-admin use, mandatory approval workflow) is a hardening layer 26_* scopes; the MVP floor is **per-query attributable audit rows**, which the policy-based bypass already guarantees.

5. Role-vocabulary reconciliation (§11.4.186 — single canonical vocabulary)

The same role vocabulary is represented in **five places** and they diverge — exactly the “more than one representation of the same tracked data” §11.4.186 forbids drifting. The divergences (12_* §6 + 23_* §5, re-verified this session):

- **Server** closed role set admin | operator | viewer | device (token.go:14-20) — **no super_admin**.
- **OTA Manager**
Role = 'viewer' | 'operator' | 'admin' | 'device' | 'super_admin' (api-client.ts:323) + ROLE_HIERARCHY with super_admin:3 (permissions.ts:12-18) — **has super_admin** (the fullest set).
- **Dashboard** Role = "admin" | "operator" | "viewer" | "device" (dashboard/src/types/api.ts:12) + TokenResponse.roles (:24) — **omits super_admin** → the super-admin console cannot be gated on the dashboard until fixed.
- **OTA Manager sidebar** nav gates on a literal **"developer"** (sidebar.tsx:20-21, roles: ["admin", "developer"]) that is **not a member of the Role union at all** — a latent nav-gate bug (RoleGate is UX-only, so not an auth hole, but those items never match by that literal).
- 20_* Membership.role CHECK IN ('viewer', 'operator', 'admin') — the **per-account membership** subset.

5.1 The single canonical role vocabulary

Two distinct axes must be named precisely (conflating them is the root of the drift):

- **Token roles claim (authentication vocabulary):** viewer | operator | admin | device | super_admin — the OTA Manager set (api-client.ts:323) is canonical. device is a token-only role (token.go:19); super_admin is **derived from the global**

users.is_super_admin flag (§1.4, 20_* §6) and surfaced as a role value **for token routing + UI gating only** — it is NOT a Membership.role.

- **Membership.role (per-account authorization vocabulary):** viewer | operator | admin (20_* §1.4) — the per-account subset; device and super_admin are deliberately absent (a device is not an account member; a super-admin is global, not per-account).

5.2 Where each surface aligns

Surface	Current	Canonical alignment
Server token.go:14-20	admin\ operator\ viewer\ device	add super_admin as a token role value derived from is_super_admin — server becomes the SSOT the SPAs mirror
OTA Manager api-client.ts:323 + permissions.ts:12-18	full set incl. super_admin	already canonical — no change
Dashboard types/ api.ts:12,24	omits super_admin	add super_admin to Role + TokenResponse.roles (23_* §5 — must precede any dashboard super-admin UI)
OTA Manager sidebar.tsx:20-21	literal "developer" (not in union)	replace "developer" with a real union role during the sweep — reconciled to the intended role (candidate: operator), not silently guessed here (§11.4.6) — flagged for the implementer
20_* Membership.role	viewer\ operator\ admin	already canonical (the per-account subset) — no change

Recommendation: make the `server token.go` role set the **single source of truth** for the authentication vocabulary; both SPAs' `Role` unions mirror it (a §11.4.186 single-representation contract), and a lint/type check (or the §11.4.186 cross-document consistency gate) fails if a surface's union diverges. Do the dashboard `super_admin` addition + the `sidebar.tsx` "developer" fix in one reconciliation sweep before any super-admin UI lands. **Tradeoff:** this is a small, low-risk type + literal change, but load-bearing — a divergent role literal is a latent gate bug (the "developer" case proves the class already bit).

6. Least-privilege + revocation

6.1 Least-privilege defaults

- **New membership defaults to viewer** (20_* §1.3 `Membership.role` DEFAULT 'viewer') — the lowest privilege; grants are explicit escalations.
- **CLI API keys are account-scoped, optionally project-narrowed** (24_* §1.1): a key binds to exactly one `account_id` and (optionally) one `project_id`; scope is **authoritative from the stored key, never asserted by the request** (24_* §1.2). NULL `project_id` = any project the caller is authorized for in that account.
- **Device tokens are scoped to (account, project, device)** (25_* §1.2), server-minted, never self-asserted.
- **Super-admin is the sole global identity** (§1.4) — everything else is scope-bounded; the tenant-isolation predicate (§1.2(1)) is the default-deny for cross-account.

6.2 Revocation semantics

Revocation event	Mechanism	Effect + latency
API key revoked (20_* <code>api_keys.revoked_at</code>)	middleware checks <code>revoked_at</code> IS NULL on every key resolution	direct-key path (24_* Option A): immediate (row checked each call). exchange path (24_* Option B):

Revocation event	Mechanism	Effect + latency
Refresh token revoked / logout	in-memory single-use-rotating store (handlers_auth.go:28-72)	the already-minted short-lived access token survives to its TTL (15 min) — bounded, not immediate (24_* con); the key can mint no <i>new</i> tokens refresh invalidated immediately; the outstanding access token expires within its short TTL the account-scoped token is rejected at the next request (not TTL-bounded) — because the claim is a hint and the membership row is the authority; a removed member cannot re-exchange for that account
Membership removed (RemoveAccountMembership, 20_* §3.1)	requireAccountAccess re-verifies the membership row per request (§2.2)	

Revocation event	Mechanism	Effect + latency
Account suspended (20_* status='suspended')	attribute deny-override (§1.2(3)) at requireAccountAccess + gate login/exchange	all sign-in, exchange, and account-scoped access fail-closed for that account immediately the session immediately re-scopes — the policy-based (not BYPASSRLS) bypass is why revocation takes effect at once (20_* §6; a bypass that survives revocation is a §11.4 bluff — 26_*'s paired-mutation proof)
Super-admin revoked (users.is_super_admin=false, config/bootstrap only)	clears the flag → clears the super-admin GUC	

6.3 Recommendation — pair short TTL with a membership/claim-version check

The only non-immediate revocation vectors above (API-key exchange-token, and generally any minted short token) are **bounded by the short access TTL** (multi-tenant-saas.com’s “short lifetime bounds the window” — §Sources). To tighten membership-removal specifically below the TTL, **recommend a per-(user, account) claim-version** (the claim_ver pattern — §Sources) reconciled by requireAccountAccess: bumping the version on membership removal/role-change rejects a removed member’s outstanding short token at the next request rather than at TTL expiry. **Tradeoff:** the version bump adds one field + one comparison; without it, the worst-case window for a removed member

is one access TTL (15 min) — small, and already the belt to the per-request membership re-verify's suspenders (§2.2). For the MVP the per-request membership re-verify alone is sufficient (removal takes effect next request); the claim-version is the hardening escalation 26_* may adopt. **Super-admin credential rotation** follows the break-glass norm (§Sources) — config-seeded, isolated, rotatable, every use audited (§4.1).

Sources verified 2026-07-10

External best-practice research per §11.4.8 / §11.4.99 — full pages fetched + cross-checked this session (not from memory); each is the ground for the section noted.

- **Auth0 — “How to Choose the Right Authorization Model for Your Multi-Tenant SaaS Application.”** <https://auth0.com/blog/how-to-choose-the-right-authorization-model-for-your-multi-tenant-saaS-application/> — verified: RBAC is the starting point; roles MUST be checked *within the tenant context* (“admin in Tenant A, viewer in Tenant B”); ABAC for dynamic/context-aware decisions; ReBAC for relationship-heavy sharing. Grounds §1.1 / §1.2 / §1.3. Negative finding (§11.4.99): this page does **not** endorse combining models in one app — the hybrid recommendation rests on the AWS + Cerbos sources, not this one.
- **AWS Prescriptive Guidance — “Multi-tenant access control with RBAC and ABAC” (Verified Permissions / Cedar).** <https://docs.aws.amazon.com/prescriptive-guidance/latest/saas-multitenant-api-access-authorization/avp-mt-abac-rbac-examples.html> — verified: the RBAC+ABAC hybrid puts a **tenant-isolation condition on every policy** (resource in principal.Tenant) composed WITH role membership AND an attribute deny (account_lockout_flag == false), over one shared multi-tenant policy store. Grounds the §1.2 3-step ordering (tenant-isolation first, then role, then attribute-deny) + §4.
- **Multi-Tenant SaaS Architecture Hub — “JWT Claims for Tenant Scoping: Best Practices.”** <https://www.multi-tenant-saas.com/auth-isolation-cross-tenant-access-control/tenant-aware-jwt-token-management/jwt-claims-for-tenant-scoping-best-practices/> — verified: use a **custom, top-level, immutable tenant_id claim** (not reused OIDC fields), validated once at the gateway and read only from the signed payload; bound staleness with **short TTL + a claim_ver version claim + key rotation**; for multi-tenant users,

one token per active tenant session OR a tenants array — **avoid a flat global role list (lateral movement)**. Grounds §3 + §6.3.

- **Britive** — “**Break Glass Account Management Best Practices**.”
<https://www.britive.com/resource/blog/break-glass-account-management-best-practices> — verified: emergency-only use, isolation from normal admin, **every use logged + alerted + audited**, eliminate standing privilege (just-in-time + rotate credentials), approval workflow, periodic testing. Grounds the §1.4 / §4.1 / §6.3 super-admin = break-glass-class treatment.
- **WorkOS** — “**How to design an RBAC model for multi-tenant SaaS**” (multi-tenant RBAC data model: membership join keyed (user, tenant, role); shared default roles + per-tenant customization) and “**Multi-tenant session management**” (one identity-scoped session; active org is separate state exchanged on demand for a short-lived access token carrying the chosen org id).
<https://workos.com/blog/how-to-design-multi-tenant-rbac-saas> + <https://workos.com/blog/multi-tenant-session-management> — verified in-doc-set (cited by 20_* and 23_*, dated 2026-07-10); reused here for the §1 role model + the §2 exchange flow (cross-document consistency, §11.4.186).

No source prescribes Helix-OTA-specific choices — the promotion of the shipped RESOURCE_PERMISSIONS grid to a server-authoritative permission set (§1.3), the belt-and-suspenders “stamp the claim AND re-verify membership per request” (§2.2), and the fail-closed legacy-token handling (§3.3) are **original work** applied to this codebase’s shape, following from the shipped token.go/permissions.ts/middleware.go seams.

Honest boundary (§11.4.6)

- **This is a design proposal, not implemented.** No claim, middleware, RLS policy, role reconciliation, or revocation mechanism described here exists in the codebase yet. Every “as-is” fact carries a file:line (grounded by 10_* / 12_* / 20_*); every “to-be” element is a proposal. Nothing here is a §11.4 completion claim.
- ****Entity shapes are 20_*’s, never this doc’s (§11.4.186).**** Account / User / Membership / api_keys / the reserved roles-permissions tables are quoted from 20_target_multitenancy_data_model.md and never redefined. Where the two docs framed the token differently, §2.2 states the reconciliation explicitly (stamp-claim-AND-verify) so they do not silently diverge — this doc designs authZ *behaviour*, 20_* owns the shapes.

- **Every open choice is a recommendation + tradeoffs for operator decision, never silently decided:** the permission-model shape (§1.2 RBAC-first hybrid vs full ABAC engine), the exchange-vs-embedded token (§2.2), the legacy-token handling (§3.3 Option A/B/C), the role-vocabulary SSOT + the "developer" fix (§5), the claim-version hardening (§6.3), and per-tenant signing keys (§3.2). Each names its cost.
- **The endpoint shapes are 22_*'s; the isolation PROOF is 26_*'s.**
This doc names a /auth/token-exchange flow and a requireAccountAccess middleware conceptually — the concrete route table, request/response DTOs, and status codes are 22_api_surface.md's; the paired-mutation proof that a non-scoped session cannot read another account's rows (L1/L2/L3 all held, both backends) is 26_security_threat_model.md's. This doc proves neither — it specifies the model they realize + test.
- **What the recommendation does NOT prove.** RBAC-first + tenant-isolation-predicate is the correct *default*, not a guarantee of isolation — isolation is only as good as the L1 ordering + the L3 policies + the scope-resolution code, which is exactly why 26_* owns the adversarial proof. The MemoryRepository MVP backend has no L3/RLS — its isolation rests entirely on L1+L2 (§4), a fact flagged, not glossed.
- **Scope of the underlying audit.** The file:line references were established against non-test source only (10_* / 12_* scope); a test path may exercise a detail not seen. The rollout subsystem's tenancy (internal/rollout/) is UNCONFIRMED: in 10_* and is not scoped here — its authZ must be audited before its routes are gated. The device-token mint call site is UNCONFIRMED: in 10_* (device claim design leans on 25_*'s reading, not a re-audit here).